

Lab 03 – Worksheet

Name: Mohammad Husain Naqvi	ID: 11062	Section: T-4
-----------------------------	-----------	--------------

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Provide appropriately commented codes (Mention question part before each part)

```
.text
.globl main
main:
    li x10, 12      # argument a
    li x11, 12      # argument b
    jal x1, sum

    mv x11, x10      # moving sum to x11 which is the argument register
    li x10, 1        # ecall print statement
    ecall
    j exit

sum:
    add x10, x10, x11 # return a + b in x10
    jalr x0, 0(x1)

exit:
    j exit
```

Add screenshot of your results (register)

Before ecall condition (li x10x1):

```
x08 (s0) = 0
x09 (s1) = 0
x10 (a0) = 12
x11 (a1) = 12
x12 (a2) = 0
x13 (a3) = 0
```

```
PC = 28
```

After Returning and ecall condition :

```
x10 (a0) = 1
x11 (a1) = 24
x12 (a2) = 0
```

```
PC = 36
```

```
> PRIV
```

Summary:

x10 = 12

x11 = 12

FUNCTION:

x10 = x10 + x11

x10 = 24

returns :

x10 = 24

Move x10 in x11:

So x11 = 24

ecall condition is li x10,1

so x10 = 1

prints: x11 which is 24.

j end.

Task 2

Provide appropriately commented codes (Mention question part before each part)

```
.text
.globl main
main:
    li x10, 4      # g
    li x11, 3      # h
    li x12, 2      # i
    li x13, 1      # j

    jal x1, leaf_example

    mv x11, x10     # move f to argument register
    li x10, 1
    j exit

leaf_example:
    addi sp, sp, -12 # making space in the stack
    sw x18, 0(sp)
    sw x19, 4(sp)
    sw x20, 8(sp)

    add x18, x10, x11 # g + h
    add x19, x12, x13 # i + j
    sub x20, x18, x19 # f = (g + h) - (i + j)
    mv x10, x20      # return f in x10

    lw x18, 0(sp)
    lw x19, 4(sp)
    lw x20, 8(sp)
    addi sp, sp, 12  # deleting the space.
    jalr x0, 0(x1)

exit:
    j exit
```

Add screenshot of your results

Initialization:

```
main:
    li x10, 4      # g
    li x11, 3      # h
    li x12, 2      # i
    li x13, 1      # j
```

```
x10 (a0) = 4
x11 (a1) = 3
x12 (a2) = 2
x13 (a3) = 1
```

Register values at the end:

```
x10 (a0) = 1
x11 (a1) = 4
x12 (a2) = 2
x13 (a3) = 1
```

$f = (g + h) - (i + j)$
 $f = (4 + 3) - (2 + 1)$
 $f = 7 - 3$
 $f = 4$

Answer = 4

which is x11 in our case.

x10 is 1 because of ecall condition. (li x10, 1)

Task 3 Provide appropriately commented codes (Mention question part before each part)

```
.text
.globl main

main:
    li x20, 0x200

    li x21, 10
    sb x21, 0(x20)

    li x21, 20
    sb x21, 4(x20)

    li x21, 30
    sb x21, 8(x20)

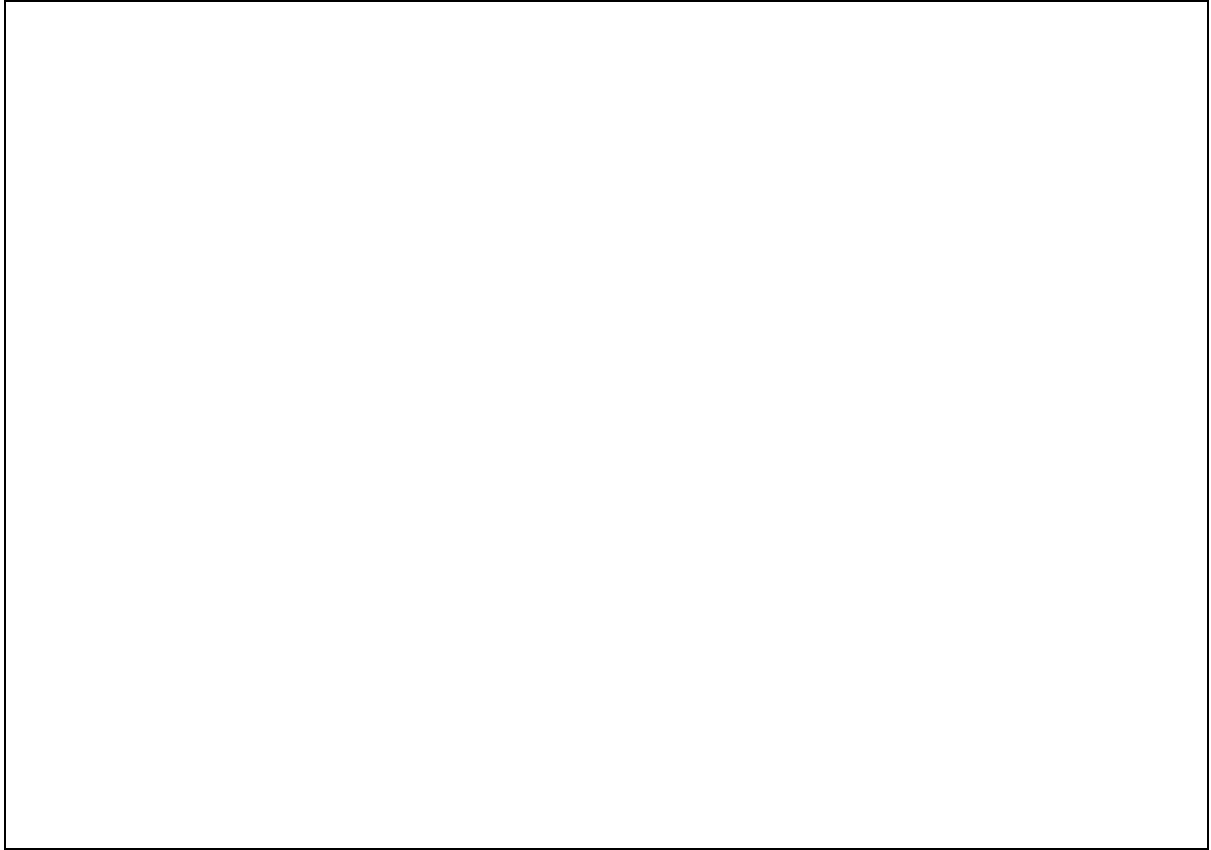
    li x21, 40
    sb x21, 12(x20)

    li x21, 50
    sb x21, 16(x20)

    li x10, 0x200    # base address of v[]
    li x11, 3        # k
    jal x1, swap
    j exit

swap:
    slli x5, x11, 2
    add x5, x10, x5    # address of v[k]
    lw x6, 0(x5)       # temp = v[k]
    lw x7, 4(x5)       # value of v[k+1]
    sw x7, 0(x5)       # v[k] = v[k+1]
    sw x6, 4(x5)       # v[k+1] = temp
    jalr x0, 0(x1)

exit:
    j exit
```



Add screenshots of your results

Before swap:

```
0x00000210      32
0x0000020C      28
0x00000208      1E
0x00000204      14
0x00000200      0A
0x000001FC      00
```

After Swap:

```
0x00000210      28
0x0000020C      32
0x00000208      1E
0x00000204      14
0x00000200      0A
0x000001FC      00
```

The values of v[3] and v[4] are swapped because k = 3.

Task 4 Provide appropriately commented codes

```
.text
.globl main
main:

    li x20, 0x200      # x20 = address of y['n', 'a', 'q', 'v', 'i']

    li x21, 110        # n = 110
    sb x21, 0(x20)

    li x21, 97         # a = 97
    sb x21, 1(x20)

    li x21, 113        # q = 113
    sb x21, 2(x20)

    li x21, 118        # v = 118
    sb x21, 3(x20)

    li x21, 105        # i = 105
    sb x21, 4(x20)

    li x21, 0          # '\0' = 0
    sb x21, 5(x20)

li x10, 0x100 # x[]
li x11, 0x200 #y[]
li x0, 0
jal x1, strcpy
li x10, 1
ecall
end:
j end

strcpy:
    li x12, 0

for:
    add x16, x10, x12 # because it's a character its a single byte so we directly add instead
of shifting.
    add x17, x11, x12

    lb x19, 0(x17) # load value form y[] onto x19 register
    sb x19, 0(x16) # transfer the value from x19 onto x[]

    beq x19, x0, Else # checking the condition if y[i] = '\0'
    addi x12, x12, 1 # i= i+1
    j for

Else:
    jalr x0, 0(x1) # returning to funciton.
```


Add screenshots of your results

Initialization array = ['n', 'a', 'q', 'v', 'i']
 Base address= 0x200 y[]

These are the ASCII codes for n , a, q, v and i as also mentioned in above code.

0x00000208	0	0	0	0
0x00000204	105	0	0	0
0x00000200	110	97	113	118
0x000001FC	0	0	0	0

Before loop:

Base address 0x100 for x[]

0x00000104	0	0	0	0
0x00000100	0	0	0	0

Loop:

i= 0

0x00000104	0	0	0	0
0x00000100	110	0	0	0

i=1

0x00000104	0	0	0	0
0x00000100	110	97	0	0
0x000000FC	0	0	0	0

i=2

0x00000104	0	0	0	0
0x00000100	110	97	113	0
0x000000FC	0	0	0	0

i=3

0x00000104	0	0	0	0
0x00000100	110	97	113	118

i=4

0x00000104	105	0	0	0
0x00000100	110	97	113	118

Basically copies values from y[] onto x[] until the value hit a '\0' or 0 in assembly.

Lab 03 – Introduction to RISC V Assembly (Jumps and Returns)

Assessment Rubrics

Points Distribution

Points Distribution			
Task No.	LR2 Code	LR5 Results	AR7 Report Submission and Git Upload
Task 1	/10	/05	/10 & /10
Task 2	/10	/10	
Task 3	/10	/10	
Task 4	/15	/10	
Total Points	/100 Points		
CLO Mapped	CLO 2		

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.